

阶段四：多卡扩展性对比报告

模型: Qwen3.5-9B (FP8 在线量化)
硬件: 2 x NVIDIA RTX 4090 24GB, PCIe 互联 (无 NVLink, SYS 拓扑)
vLLM: 0.17.0
日期: 2026-07-09

一、实验设计

1.1 目标

在 2x4090 (PCIe-only, 无 NVLink) 环境下, 对比三种部署拓扑的吞吐与延迟表现:

配置	启动参数	说明
单卡 baseline	CUDA_VISIBLE_DEVICES=0	只用一张卡, 其余留空
TP=2	--tensor-parallel-size 2	权重切分到两张卡, 每生成一个 token 触发 all-reduce
PP=2	--pipeline-parallel-size 2	模型按层切分到两张卡, 每次前向只做一次点对点传输

1.2 压测参数

- 数据集: 100 条混合 prompt (短/长/代码/创意各 20 条), 循环展开
- 精度: FP8 在线量化 (--quantization fp8), 三种配置统一精度
- 指标: 墙钟吞吐 (t/s)、端到端延迟 P50/P99、TTFT P50、TPOT P50
- 并发梯度: 1 / 4 / 8 / 16 / 32 / 64

1.3 硬件拓扑

```
nvidia-smi topo -m 输出:  
GPU0 <-> GPU1: SYS (跨 NUMA 节点, PCIe + QPI/UPI 互联)
```

这是通信条件最差的场景之一, all-reduce 和 P2P 带宽均受限。

二、三种配置核心数据对比

2.1 墙钟吞吐 (tokens/s)

并发	单卡	TP=2	PP=2	TP/单卡	PP/单卡
1	72.7	121.4	72.5	1.67x	1.00x
4	286.4	481.9	287.4	1.68x	1.00x
8	514.7	833.9	516.4	1.62x	1.00x
16	1015.8	1481.1	1016.6	1.46x	1.00x
32	1633.0	2168.5	1632.3	1.33x	1.00x
64	2667.4	3312.0	2668.6	1.24x	1.00x

2.2 端到端延迟 P50 (秒)

并发	单卡	TP=2	PP=2
1	6.62	3.78	6.55
4	6.88	4.09	6.86
8	7.15	4.46	7.13
16	7.73	5.27	7.69
32	9.06	6.94	9.10
64	12.37	10.08	12.45

2.3 TTFT / TPOT (秒)

并发	单卡 TTFT	TP=2 TTFT	PP=2 TTFT	单卡 TPOT	TP=2 TPOT	PP=2 TPOT
1	0.072	0.045	0.058	0.013	0.007	0.013
4	0.101	0.072	0.113	0.013	0.008	0.013
8	0.099	0.077	0.118	0.014	0.009	0.014
16	0.187	0.156	0.166	0.015	0.010	0.015
32	0.265	0.224	0.223	0.017	0.013	0.017
64	0.339	0.365	0.306	0.023	0.018	0.023

三、关键发现与分析

3.1 TP=2：加速有效，但通信损耗随并发增大

- 低并发 (c1-c8)：效率 81-84%，每张卡的算力基本被利用，all-reduce 开销占比小
- 高并发 (c32-c64)：效率降到 62-66%，并发越大每张卡的 batch size 越小，通信/计算比上升，all-reduce 成为瓶颈
- TPOT 显著降低：c1 时 TPOT 从 0.013 降到 0.007 (快 46%)，证实并行计算确实加速了 token 生成
- TTFT 在高并发反超：c64 时 TPOT 0.365 > 单卡 0.339，all-reduce 的累积延迟在排队场景下暴露

机制解释：TP 每生成一个 token 都要做 all-reduce (聚合激活值)。无 NVLink 时走 PCIe，带宽约 16GB/s (vs NVLink 400GB/s)，高频 all-reduce 的累积延迟在高并发下不可忽略。

3.2 PP=2：几乎零加速，pipeline bubble 抵消并行收益

- 所有并发下 PP/单卡比值稳定在 1.00x，效率约 50%
- 延迟与单卡几乎完全一致：P50 差异 < 3%
- TPOT 与单卡完全相同：0.013 vs 0.013，说明串行执行没有带来计算加速

机制解释：PP 把模型按层切成两半，GPU0 跑前半层 → 传中间结果给 GPU1 → GPU1 跑后半层。每步只做一次 P2P 传输，通信量比 TP 少得多。但问题是：

1. Pipeline bubble: GPU0 传完数据后必须等 GPU1 处理完才能传下一条，空闲等待时间长
2. PCIe 延迟: P2P 走 PCIe，延迟数百微秒，9B 模型每层激活值不小，传输时间累加
3. 单卡本就能跑: 9B FP8 约 9.5GB 权重，单卡 24GB 绰绰有余，PP 的切分没有解决显存瓶颈，反而引入了 pipeline 开销

3.3 核心工程结论

吞吐排序 (64 并发):	
2 个单卡实例 (独立服务)	≈ 5334 t/s (理论)
TP=2	= 3312 t/s (+24% vs 单卡, 但远低于 2x)
单卡 baseline	= 2667 t/s
PP=2	= 2669 t/s (≈ 单卡, 没有提升)

决策建议:

1. 单卡能装下的模型 (如 9B), 在 PCIe-only 环境下:
- 优先跑多个独立单卡实例, 吞吐线性叠加, 无通信开销

• TP=2 只在低并发/延迟敏感场景有价值 (TTFT/TPOT 可降低 40-50%)

• PP=2 对这种规模模型几乎无益
2. TP vs PP 选型原则:
- 有 NVLink → TP 是首选, all-reduce 带宽充足, 效率可达 90%+

• 无 NVLink + 模型单卡装得下 → 跑 N 个单卡实例 > 任何 N 卡并行方案

• 无 NVLink + 模型单卡装不下 → PP 比 TP 更省通信, 但吞吐提升有限, 需接受延迟换容量的 trade-off

四、面试话术要点

1. “为什么双卡 TP 没有 2 倍加速?”

TP 每生成一个 token 都要 all-reduce, 无 NVLink 时走 PCIe, 带宽只有 16GB/s 级别。并发越高 batch size 越小, 通信/计算比越大, 效率从 84% 降到 62%。这是分布式推理的经典通信瓶颈。

2. “PP 为什么完全没有提升?”

PP 的加速前提是模型大到单卡装不下, 必须切层。9B FP8 只有 9.5GB, 单卡 24GB 很充裕, PP 的切分反而引入了 pipeline bubble 和 P2P 延迟, 没有解决任何瓶颈。

3. “实际部署怎么选?”

先算显存: 单卡装得下 → 跑多个单卡实例; 装不下 → 看互联带宽。有 NVLink 用 TP, 只有 PCIe 用 PP 保底。不要假设“加卡一定更快”, 要先验证通信开销是否值得。

4. “怎么识别配置错误导致的‘双卡比单卡慢’?”

常见坑是给双卡设 TP=1 (没真正开并行, 只是内存共享), 或者 PP=2 但 batch size=1 导致 bubble 占比 100%。正确做法是: TP=2 要确认 tensor_parallel_size=2 生效, PP=2 要保证 batch size >= PP stage 数, 让 pipeline 填满。

五、实验环境快照

项目	值
vLLM	0.17.0
PyTorch	2.10.0
Transformers	4.57.6
GPU	2 x RTX 4090 24GB

Table 5 – continued

项目	值
GPU 互联	PCIe SYS (无 NVLink)
CUDA	12.8
模型	Qwen/Qwen3.5-9B
量化	FP8 在线量化
max_model_len	8192

六、阶段一到四完整链路

阶段	内容	状态
一	基线服务搭建 (单卡 FP16)	已完成
二	压测体系构建 (流式 TTFT/ TPOT)	已完成
三	量化对比 (FP16 vs AWQ vs FP8)	已完成
四	多卡拓扑对比 (单卡 vs TP=2 vs PP=2)	已完成

阶段四数据直接呼应阶段三的发现 (FP8 是” 全优生”), 并为阶段五 (多实例部署) 提供选型依据。